

■ SERPENT v5

AI-Assisted Game Development Prompt Engineering Process Report

Project	SERPENT v5 – Cyberpunk Snake Game
Developer	RaiedOP
Tool Used	Claude (Anthropic AI)
Version	v5 (Final)
Technologies	HTML5 · CSS3 · JavaScript · Three.js

This report documents the iterative AI prompt engineering process used to design, develop, and refine SERPENT v5 — a feature-rich cyberpunk snake game — entirely through conversations with the Claude AI assistant.

1. Project Overview

SERPENT v5 is a browser-based snake game built entirely with HTML, CSS, and JavaScript using the Three.js library for 3D rendering. The game features four distinct play modes (Classic, Maze, Endless, and Ghost), a cyberpunk aesthetic with animated backgrounds, a full in-game skin shop with eight purchasable skins, a persistent leaderboard, combo scoring, wave mechanics, mobile touch controls via an on-screen D-pad, and particle effects per skin. Every line of code, every design decision, and every feature iteration was produced through structured AI prompting — no external tutorials or copied code were used.

Features	Description
4 Game Modes	Classic · Maze · Endless · Ghost
8 Skins	Cyber, Plasma, Venom, Inferno, Aurum, Phantom, Toxic, Void
3D Rendering	Three.js with perspective camera and lighting
Coin Economy	Earn coins, spend in shop, persistent via localStorage
Leaderboard	Per-mode high scores stored locally
Mobile Support	Touch swipe + on-screen D-pad
Visual FX	Particle effects, matrix rain, scanlines, glitch animations
HUD	Live score, level, coins, speed bar, combo flash

2. Prompt Iterations Used During Development

The game was developed across five major prompt iterations. Each iteration built upon the previous version, adding features, fixing issues, and refining mechanics. Below is a detailed walkthrough of how each prompt was structured and what it achieved.

Iteration 1 — Core Game Foundation (v1)

Prompt:

"Create a snake game in a single HTML file using JavaScript. The snake should move on a grid, eat food to grow, and end the game when it hits a wall or itself. Style it with a dark cyberpunk theme."

Goal:

Establish a working baseline game loop — movement, collision detection, food spawning, and score tracking — wrapped in a visually styled HTML file.

What Was Produced:

- A single-file HTML game with a 2D canvas grid
- Arrow key controls for snake movement
- Basic food spawning and score counter
- Dark background with neon green snake styling
- Simple game-over detection (wall and self-collision)

Iteration 2 — 3D Upgrade & Multiple Modes (v2–v3)

Prompt:

```
"Upgrade the snake game to use Three.js for 3D rendering. Add four game modes: Classic (speed scales with level), Maze (static obstacles), Endless (speed spikes every 10 seconds), and Ghost (walls wrap, no self-collision). Show a mode selection screen at the start."
```

Goal:

Elevate the visual presentation to 3D and diversify the gameplay by introducing distinct mode-specific rule sets.

What Was Produced:

- Three.js perspective camera with grid floor and ambient lighting
- 3D snake body segments rendered as box geometries
- Mode selection overlay with four styled cards
- Classic mode: level system scaling tick speed (BASE_TICK=155ms → MIN_TICK=60ms)
- Maze mode: 26 randomly placed wall obstacles
- Endless mode: wave-based speed spikes every 10 seconds
- Ghost mode: toroidal (wrap-around) movement, no self-collision

Iteration 3 — Economy, Shop & Persistence (v3–v4)

Prompt:

```
"Add a coin economy: the snake earns coins when eating food (3 coins normal, 9 for bonus food). Add a Skin Shop with 8 purchasable skins, each with a unique colour and particle effect label. Save coins, owned skins, equipped skin, and best scores to localStorage so they persist between sessions."
```

Goal:

Introduce a progression and monetisation loop that gives players a reason to keep playing, and ensure all progress persists across browser refreshes.

What Was Produced:

- Coin counter in HUD; coins awarded on food eaten
- Bonus food mechanic with 22% spawn chance and 6.5s timer
- Shop overlay with 8 skin cards (Cyber, Plasma, Venom, Inferno, Aurum, Phantom, Toxic, Void)
- Each skin defined with head/body/trail colours and a particle FX tag
- localStorage save/load for coins, best score, owned skins, and current skin
- Shop accessible from the start screen, HUD, pause menu, and game-over screen

Iteration 4 — Visual Polish & UX Refinement (v4)

Prompt:

```
"Add a dramatic animated intro screen that types out 'SERPENT' letter by letter with glitch effects. Add a matrix rain background canvas layer. Create a HUD with score, coins, level, and a speed bar at the bottom. Add combo scoring (eating multiple foods quickly multiplies points). Show wave flash messages. Make all UI fully responsive for mobile screens."
```

Goal:

Transform the game from functional to polished — delivering a premium feel through animation, responsive layout, and satisfying feedback loops.

What Was Produced:

- Typewriter intro animation for the SERPENT title with CSS glitch keyframes
- Matrix rain canvas layer (z-index 1) with fading columns of characters
- Animated scanline overlay during intro sequence
- Pill-shaped HUD elements with neon glow and backdrop blur
- Speed bar (bottom of screen) filling as level increases
- Combo flash overlay: x2/x3 multiplier shown at screen centre
- Wave flash: 'WAVE 2 – SPEED SURGE' style messages
- Full CSS clamp() responsive sizing for all fonts and padding
- On-screen D-pad shown on touch/mobile devices via CSS media queries

Iteration 5 — Final Integration & Bug Fixing (v5)

Prompt:

```
"Review the full game for bugs. The leaderboard should show per-mode best scores with gold/silver/bronze rank styling. The pause screen should work in all modes. Touch swipe detection should not conflict with the D-pad. Ensure the shop previews render correctly. Fix any rendering glitches on mobile. This should be the final polished v5."
```

Goal:

Consolidate all features into a bug-free, shippable final version with leaderboard, proper pause handling, and conflict-free input systems.

What Was Produced:

- Leaderboard overlay with per-mode entries, gold/silver/bronze rank colours
- Pause screen accessible via HUD button or 'P' key in all modes
- Swipe gesture detection (touchstart/touchend) running independently from D-pad tap events
- Shop skin preview bars rendered as coloured rectangles via canvas context
- Input queue (iq[]) to buffer direction changes and prevent 180-degree reversals
- Complete localStorage schema: { c: coins, b: best, s: skin, o: owned }
- Responsive overflow handling for shop grid and leaderboard list

3. What Worked Well

✓ Incremental Feature Addition

Breaking the project into focused prompts — each targeting one system (rendering, economy, UI, bug fixes) — worked extremely well. Claude could hold context about the existing code and extend it cleanly without rewriting unrelated parts.

✓ Specific Technical Requests

Prompts that named exact libraries (Three.js), exact values (BASE_TICK=155ms), and exact behaviours (wrap-around walls in Ghost mode) produced precise, correct code on the first attempt. Specificity dramatically reduced the need for corrections.

✓ CSS-First Responsiveness

Asking Claude to use CSS clamp() for all sizing and media queries for the D-pad resulted in a layout that worked correctly on both desktop and mobile without any manual tweaking afterward.

✓ Consolidated Single-File Output

Requesting everything in one .html file made the deliverable immediately testable — open in a browser and play. This fast feedback loop helped identify issues quickly.

✓ Mode Architecture

Defining all four modes upfront in a single prompt let Claude design a unified game-state system (the gs variable and mode variable) that handled all modes elegantly through shared logic with per-mode branches.

✓ Visual Theming via CSS Variables

Establishing the colour palette as :root CSS variables (--c1 cyan, --c2 magenta, etc.) early meant all subsequent visual elements automatically matched the theme without re-specifying colours in every prompt.

4. What Did Not Work

X Vague Aesthetic Prompts

"Make it look cool" or "add nice animations" produced generic results. Claude needs explicit descriptions — naming specific effects like "matrix rain", "glitch keyframes", or "scanline overlay" produced exactly what was intended.

X Asking for Too Much at Once

One early attempt combined 3D rendering, the shop, the leaderboard, mobile controls, and the intro animation into a single prompt. The result had incomplete implementations of each feature and several bugs. Splitting concerns into separate prompts fixed this.

X Assuming Context Persists

In longer sessions, Claude occasionally lost track of earlier decisions (such as the exact skin IDs or the localStorage key names) when the conversation grew very long. Pasting the current code back into the prompt as context resolved this.

X Touch & Keyboard Input Conflicts

The first version of mobile input used touchstart on the whole document, which conflicted with button taps in the HUD. The fix required an explicit prompt specifying that D-pad button presses should use pointer events, not touch events.

X Three.js Version Mismatch

An early prompt requested features from a newer Three.js API (OrbitControls, CapsuleGeometry) while the CDN link pointed to r128. This caused silent failures. Specifying the exact CDN version (r128) in the prompt resolved the issue.

X Particle Effects Not Implemented

The skin shop lists particle effect labels (e.g., '■ Fire embers') but the actual particle systems were not fully implemented in v5 — they are described but not rendered in-game. This was an oversight from keeping scope manageable.

5. How Prompts Were Improved Over Time

Prompt quality improved significantly across the five iterations. The table below contrasts early-stage prompts with the refined versions used in later iterations.

Aspect	Early Prompt (v1–v3)	Improved Prompt (v4–v5)
Specificity	"Make it look cyberpunk"	"Use #00f5ff cyan for accents, add a matrix rain canvas layer behind the UI"
Scope	"Add shop and leaderboard and mobile support"	"Add only the skin shop this iteration; mobile and leaderboard come in a later iteration"
Library versions	"Use Three.js"	"Use Three.js r128 from cdnjs; avoid CapsuleGeometry and OrbitalMotion"
Context	No code included in prompt	Pasted the full current HTML at the top of the prompt for reference
Error fixing	"Fix the bugs"	"The D-pad tap events fire before the game processes them — not after"
Output format	Not specified	"Output the complete single-file HTML, no truncation, ready to save"

Key Principles Learned

- One concern per prompt — never combine unrelated features in a single request.
- Always specify library versions when using CDN-hosted dependencies.
- Include the current working code in the prompt when making targeted edits.
- Describe visual effects by name (glitch, scanline, matrix rain) not by impression (cool, fancy).
- End each prompt with an explicit output instruction: 'Output the complete file, no placeholders.'
- After each iteration, test in a browser and note specific bugs before the next prompt.

6. Challenges Faced During the Process

■ Managing Growing Code Complexity

By v4, the HTML file exceeded 1000 lines. Sending the full file back in the prompt for each edit consumed significant context space. The solution was to isolate the specific function or section that needed changing and only paste that portion, then ask Claude to return just the modified section for manual integration.

■ Keeping Feature Scope Under Control

It was tempting to add features continuously — networking, multiplayer, procedural level generation. Each new idea added complexity and risk of breaking existing systems. Discipline in limiting each iteration to a single defined goal was essential.

■ Debugging Without Direct Code Execution

Claude cannot run the code it writes. All testing had to be done manually by opening the HTML file in a browser. Bugs sometimes required multiple prompt-edit-test cycles before being fully resolved, particularly for timing-sensitive issues like the combo multiplier window (1000ms after last food eaten).

■ Mobile Testing Limitations

Responsive design had to be tested on real devices after each iteration. Some CSS `clamp()` values that looked correct on desktop produced oversized D-pad buttons on certain phone screen heights, requiring iterative adjustment.

■ Three.js Learning Curve in Prompts

Three.js has a large API surface. Early prompts used vague descriptions of 3D features, which sometimes led Claude to use deprecated or unavailable methods. Learning to reference Three.js by feature name (`BoxGeometry`, `MeshStandardMaterial`, `DirectionalLight`) rather than by desired visual outcome greatly improved accuracy.

■ localStorage Conflicts Across Sessions

During development, stale `localStorage` data from earlier versions sometimes caused the game to load corrupt state (e.g., owned skins referencing IDs that no longer existed). Adding a `try/catch` wrapper around the `load()` function and defaulting gracefully on error resolved this.

7. Reflection & Conclusion

Developing SERPENT v5 entirely through AI prompt engineering was a genuinely effective workflow for building a feature-rich browser game. The final product — a cyberpunk snake game with 3D rendering, four modes, a skin shop, coin economy, leaderboard, mobile controls, and polished animations — demonstrates that complex, production-quality web applications can be created without manually writing code, provided the prompting process is structured and iterative.

The most valuable lesson from this process is that prompt quality scales directly with output quality. Vague prompts produce vague results; specific, technically grounded prompts produce specific, working code. The discipline of breaking large goals into focused single-concern prompts, testing after each step, and carrying working context into the next iteration is what makes AI-assisted development reliable rather than random.

For future projects, the workflow would be further improved by maintaining a dedicated 'state document' — a running summary of all decisions, variable names, and architecture choices — that can be pasted into each new prompt to ensure consistent context regardless of conversation length.

Metric	Result
Total Prompt Iterations	5 major / ~20 micro-corrections
Lines of Final Code	~1020 (HTML + CSS + JS combined)
Features Implemented	12 major feature systems
Bugs Encountered	6 documented; all resolved
Time to v5	Across multiple sessions over development period
Final File Format	Single self-contained .html file

Report prepared by RaiedOP · SERPENT v5 · Prompt Engineering Process Documentation